

Anas Shaaban

Efficient and Scalable UIO Sequence Generation for
Deterministic Finite State Machines

B.Sc. Computer Science

24/04/2026

Declaration

I certify that the material contained in this dissertation is my own work and does not contain unreferenced or unacknowledged material. I also warrant that the above statement applies to the implementation of the project and all associated documentation. Regarding the electronically submitted work, I consent to this being stored electronically and copied for assessment purposes, including the School's use of plagiarism detection systems in order to check the integrity of assessed work. I agree to my dissertation being placed in the public domain, with my name explicitly included as the author of the work.

Name: Anas Shaaban

Date: 24/04/2026

Abstract

This project investigates the construction of Unique Input/Output (UIO) sequences for deterministic Finite State Machines and introduces new approaches aimed at improving efficiency and scalability for large FSMs. Existing approaches are computationally expensive due to repetitive evaluation, exponential growth of the search space, and limited use of parallelisation, which reduces their practicality.

To address this, multiple algorithms were designed and implemented, including several variants of the newly proposed ASPR family, GA Block and GA Table. These algorithms aim to reduce unnecessary computation by limiting repeated evaluations, introducing structured state partitioning and the use of parallel execution to better utilise available hardware resources.

Experiments were conducted on each algorithm with multiple FSMs, each with varying numbers of states, inputs, and outputs. The results show that the proposed approaches are significantly computationally faster and scale better compared to traditional methods.

Contents

1 Introduction	4
2 Preliminaries	6
3 High-Level Design	9
3.1 ASPR Family	9
3.1.1 ASPR Standard	9
3.1.2 ASPR Full	10
3.1.3 ASPR Hybrid	10
3.3 GA-Block	11
3.4 GA Table	11
3.4.1 GA Table Variants	11
3.5 Parallel Variants	12
4 Low-Level design	13
4.1 Genetic Algorithm	13
4.2 ASPR Family	14
4.2.1 ASPR Standard	14
4.2.2 ASPR Full	15
4.2.3 ASPR Hybrid	16
4.3 GA-Block	16
4.4 GA-Table	17
4.5 Parallel Variants	19
5 Empirical Study	20
5.1 Experimental Setup	20
5.2 FSM Data	20
5.3 Evaluation Approach	21
5.4 Results	21
6 Conclusions	22
7 References	23

1 Introduction

The overall aim of this project is to design and evaluate efficient algorithms for constructing Unique Input/Output (UIO) sequences for Finite State Machines, with a particular focus on improving scalability through parallel and multithreaded approaches.

When creating software or systems, a developer typically encounters issues along the way that can affect the correctness or functionality of the final product. These can include internal errors such as syntax errors, semantic errors, runtime errors, as well as external events such as network errors or unexpected user actions. Thus, a very important part of creating the software or system is to test that the final product that is deployed functions as the developer desires. Testing can be time-consuming and resource-intensive, particularly when performed manually or on a large scale. This has led to significant interest in test automation, one important approach being model-based testing (MBT), in which tests are derived from a model of the system under test (SUT) and used to check conformance with the specification [7], [12]. A Finite State Machine (FSM) is a mathematical abstraction used to describe a system [1] and is one such model used for the SUT.

Finite State Machines (FSMs) have been widely used to model systems across multiple fields, including software engineering [3], and networking and communication protocols [10], [11], [13]. MBT is typically carried out on FSMs by applying sequences of inputs to the system under test (SUT) and verifying that the resulting output sequences match the expected behaviour [7], [12].

There are many methods of doing this, the most well-known ones being Distinguishing Sequence (DS), Adaptive Distinguishing Sequence (ADS), Unique Input Output Sequence (UIO), and Characterizing Set (W-set) [2], [3], [4]. Ideally, an FSM has a DS, as when it exists, it can identify all states using a single input sequence [2]. However, most FSMs do not have a distinguishing sequence, and so a UIO is usually used [4].

A UIO is a pair of input and output sequences, such that when the input is applied to all states, only one of them produces the output in the UIO [4]. Thus, it can be used to distinguish a state from all other states. This is useful in testing, as when it is applied to a SUT, it allows a tester to check that the SUT really is in the state that it is thought to be in [7]. Although not all FSMs have a UIO for each state, it has been reported that in practice most FSMs do have such UIOs [4], [10], and this has led to the development of many FSM-based test generation methods that use UIOs [10], [14]–[17].

The problem of checking the existence of a UIO has been proved to be PSPACE-hard, and there are no known polynomial time algorithms that construct UIOs, and there is no polynomial upper bound on UIO length [20]. Because of this, constructing UIOs can be very computationally expensive for large FSMs. Over the years, there have been many algorithms proposed that rely on single-thread approaches [14]–[17], but these are bounded by the sequential execution of the CPU. In contrast, by using a multithreaded approach that is suitable for GPUs, which assigns individual threads to explore candidate sequences or construct UIOs concurrently, it is possible to significantly accelerate the search process.

With the publication of the Compute Unified Device Architecture (CUDA) development toolkit, which allows GPU programming in a C-like language, the use of GPUs has been extended to a range of application domains [21]. This trend is expected to continue as CUDA-enabled hardware and software evolve. Modern NVIDIA GPUs that support CUDA, including cost-efficient devices such as the L4 or consumer-grade RTX 4060-class GPUs, provide thousands of CUDA cores and high memory bandwidth, enabling massive data-parallel execution.

As far as known, only one research paper, which was created by Hierons and Türker [21] proposes a parallel UIO construction algorithm. The proposed algorithm is called P-UIO, uses CUDA and is significantly faster than previous approaches. It is faster than Naik’s approach [8] by a factor of 11000 on average for most tested FSMs, and faster by a factor of 512 on average for the larger FSMs tested. Even though these improvements are huge, this algorithm does not reduce the underlying exponential search space, limiting its scalability for large FSMs. Additionally, this algorithm only makes use of

multiple GPU threads, not multiple CPU threads, which may reduce flexibility compared to methods that can efficiently utilize multi-core CPU architectures. These issues motivate the need for approaches that reduce unnecessary exploration while effectively taking advantage of available hardware resources.

The project has the following objectives:

- Design new UIO construction algorithms that require less computation or overhead compared to existing methods.
- Explore parallel based implementations including ones that use CUDA. However, testing is only done on CPU, as a CUDA enabled GPU was not available.
- Evaluate performance across FSMs with varying numbers of states, inputs, and outputs.
- Compare proposed methods with existing approaches.

This paper is organized as follows. Section 2 introduces the terminology used and reviews previously devised UIO generation techniques. Section 3 provides an overview of all the UIO construction algorithms that have been created in the project and describes their high-level design. Section 4 provides the low-level design of these algorithms. In Section 5, the experiments designed to evaluate the proposed UIO construction algorithms are explained, and the results of these experiments are provided. Finally, in Section 6 the conclusion of the experiments is given and possible lines of future work are provided.

2 Preliminaries

A Finite State Machine (FSM) is typically either a Mealy or Moore FSM [1], [3]-[5]. This project deals only with Mealy FSMs, which are modelled as a tuple $M = (S, X, Y, \delta, \lambda)$ where:

$S = \{s_1, s_2, \dots, s_n\}$ is a finite set of states, $X = \{x_1, x_2, \dots, x_r\}$ is a finite set of inputs,

$Y = \{y_1, y_2, \dots, y_v\}$ is a finite set of outputs, $\delta : S \times X \rightarrow S$ is the transition function,

$\lambda : S \times X \rightarrow Y$ is the output function [1]-[4].

FSMs are either deterministic or non-deterministic [3]-[5]. A Deterministic Finite State Machine (DFSM) is a machine that, upon receiving an input, transitions from one state to another and produces a single output. A Non-Deterministic Finite State Machine (NDFSM) is a machine that, upon receiving an input, may transition to one or more possible states. This makes testing and state-identification problems more difficult [3], [4]. This project deals only with DFSMs.

A minimal (or reduced) FSM is one such that for every pair of distinct states $s_i, s_j \in S$, there exists an input sequence $x \in X^*$ such that $\lambda(s_i, x) \neq \lambda(s_j, x)$ [3], [4]. This means that every pair of states is distinguishable by at least one input sequence. Conversely, an FSM that contains a pair of states that cannot be distinguished by any input sequence is referred to as a non-minimal (or non-reduced) FSM [3], [4].

An FSM is said to be completely specified if for every state $s \in S$ and every input $x \in X$, both the next state $s' = \delta(s, x)$ and the output $o = \lambda(s, x)$ are defined [3]-[7]. If there exists a state s and input x for which either the next state $s' = \delta(s, x)$ and the output $o = \lambda(s, x)$ is undefined, the FSM is referred to as a partially specified (or incomplete) FSM [3], [4], [6].

Finite State Machines (FSMs) are considered strongly connected if every state is reachable from every other state. By contrast, an FSM is initially connected if every state is reachable from the initial state [1], [4].

The algorithms used in this project assume that all FSMs are CDRS machines, meaning that they are completely specified, deterministic, reduced, and strongly connected [3], [4], [8]. FSMs are commonly assumed to be CDRS machines in FSM-based testing to ensure that all states are distinguishable and all transitions are well-defined [3], [4].

Suppose we have an FSM M . If M is in state $s \in S$ and an input $x \in X$ is applied, then M transitions to state s' and produces output o . The transition function is given by $\delta(s, x) = s'$, and the output function by $\lambda(s, x) = o$ [1], [2], [3]. Such a transition can be denoted by $\tau = (s, x/o, s')$, where x/o is the label of τ , s is the start state, and s' is the end state [2]-[4]. Juxtaposition is used to denote concatenation, such that if x_1, x_2 , and x_3 are inputs then $x_1x_2x_3$ is an input sequence. The symbol ε is used to denote an empty sequence. An input/output sequence consists of a sequence of input/output pairs of the form $x_1/o_1 x_2/o_2 \dots x_n/o_n$ [1], [3], [4]. Two states s_i and s_j are said to be equivalent if, for all possible input sequences, they produce identical output sequences [3], [4].

A Distinguishing Sequence (DS), also known as a preset distinguishing sequence, is a sequence of inputs which produces unique outputs for each state, thus distinguishing all states from one another [3], [4], [9]. This distinguishing sequence can be applied to any state in a SUT, and the resulting output can be checked against the specification [3], [4]. Hennie showed that if a DS exists, then a complete conformance test that checks all states of the FSM can be constructed algorithmically [9]. Subsequent research further formalized and extended these ideas, including methods for generating DS-based test sequences and analysing their limitations in practical systems [4], [10], [11]. While a DS is ideal, it rarely exists in practice [3], [4], [11].

An alternative to a distinguishing sequence is an Adaptive Distinguishing Sequence (ADS), which is similar to a DS, but instead of relying on a single preset input sequence, an ADS selects the next input based on the previous outputs observed so far [3], [4], [12]. Since a DS is a special case of an ADS, an

FSM that has a DS also has an ADS [3], [4]. Determining whether an FSM has an ADS can be done in polynomial time, and if one exists it can be constructed in polynomial time [3], [4]. However, not every FSM has an ADS [3], [4].

A Characterising Set (CS) is a set of input sequences, where each sequence distinguishes some pairs of states, but together they distinguish all pairs of states [3], [4]. Every minimal FSM with states has a CS with at most sequences of length at most , and such a CS can be found in polynomial time [3], [4], [13]. If every input sequence in is executed from state , the set of output sequences can be used to identify [3], [4]. However, characterising sets can sometimes contain very long test sequences [13]. Additionally, in most test generation techniques, in order to distinguish all states, a characterising set may result in many test sequences [4], [13]. A CS usually requires resetting the system multiple times between test sequences, and this has been found to be expensive [14], [15], [16], [17]. Because of the cost of using a CS to identify states, it is preferable to use DS or UIO sequences where they exist [3], [4]. However, if a UIO or DS is longer than an upper bound, then it may be preferable to use a characterising set, as the size of a characterising set is always [3], [4].

It has been observed that a full characterising set is not always necessary to uniquely identify a state in an FSM [3], [4], [18]. This has led to the notion of state identifiers, also referred to as a separating set. For a state in FSM , a state identifier is a set of input sequences that distinguish from all other states in the FSM [3], [4], [13]. The use of state identifiers can significantly reduce the size of the resulting sequences compared to characterising sets [3], [4], [13].

A Unique Input/Output (UIO) sequence is an input sequence that, when applied to a state, produces a unique output such that applying the input sequence to any other state produces a different output [3], [4], [8], [10], [19]. This can be used to identify a specific state [3], [4]. The construction and verification of such sequences form the core problem addressed in this work [3], [4], [8].

Initially, UIO identification focused on exhaustive and tree-based search techniques. This approach starts by applying each input, while storing the output responses of all states [3], [4]. There are two main ways to do this, namely breadth-first search (BFS) and depth-first search (DFS) [3], [4]. BFS always finds the shortest UIO sequence but usually takes longer than DFS to find all UIOs. Both these techniques are guaranteed to find a UIO if one exists, but in the worst case they both have exponential time complexity because the number of input sequences grows exponentially with sequence length [3], [4]. As a result, exhaustive methods are only practical for very small FSMs.

This is the pseudocode for BFS:

```

For each input  $x \in X$  do
  For each state  $s \in S$  do
    Compute  $\lambda(s, x)$ 
  End for

  Group states by identical output sequences

  If a group contains exactly one state  $s$  then
     $x$  is a UIO for state  $s$  End if

  If UIOs have been found for all states then
    Terminate End if
End for

```

Another approach to UIO identification was proposed by Naik [8], which incrementally constructs input sequences by partitioning states based on their observed outputs. In this technique, initially, all input sequences of length 1 are applied to all states, and states are grouped according to the output sequences

they produce. The next input sequences are created by appending each possible input to the current candidate sequences and selecting those that maximise the refinement of the state partition [8]. This process is repeated until, eventually, a UIO for a target state is obtained. This approach avoids exploring some unnecessary input sequences compared to brute-force enumeration, but can still scale poorly in the worst case [3], [4], [8].

Later, genetic algorithm (GA) approaches were proposed for UIO identification. In this approach, candidate input sequences are represented as chromosomes, a fitness function evaluates how well each chromosome distinguishes states, and evolutionary operators such as selection and mutation are used to guide the search towards promising UIO candidates [20]. Since genetic algorithms are stochastic search techniques, they are not guaranteed to find a UIO even if one exists, and they are also not guaranteed to find the shortest UIO sequence [20].

Hierons and Türker proposed a parallel algorithm called P-UIO for constructing UIO sequences [21]. Unlike exhaustive techniques such as BFS and DFS, which explore candidate sequences sequentially, P-UIO evaluates many candidate input sequences concurrently and groups states according to the output sequences they produce [21]. If a group contains only one state, then the corresponding input sequence is a UIO for that state. Candidate sequences that do not partition the states sufficiently can be discarded, while the remaining candidates are extended [21]. Although the worst-case search space remains exponential in the length of the input sequences, the practical performance is substantially improved through parallel execution [21]. Experimental results reported in that work showed average speedups of around 11000 over the compared sequential approach [21].

2.1 Summary

The preliminaries define the formal structure and properties of Mealy FSMs under CDRS assumptions, ensuring that all states are distinguishable and transitions are well-defined. Existing UIO identification approaches, including exhaustive, heuristic, and parallel methods were discussed. The next chapter provides the high-level design of the proposed algorithms for this project.

3 High-Level Design

In this section the high-level design of multiple proposed algorithms is provided, along with the design decisions that guided their development, their benefits and trade-offs. 2 types of algorithms were proposed, the first named Adaptive State Partition Refinement (ASPR) and the second called GA-based algorithms. The following subsections will go over the high-level design of both the ASPR and the new genetic algorithm approaches.

3.1 ASPR Family

To address the limitations of existing approaches, a family of algorithms called Adaptive State Partition Refinement (ASPR) was developed. ASPR aims to improve scalability compared to traditional approaches by reducing unnecessary computation and allowing more efficient use of hardware resources.

Two limitations of GAs formed the basis of the design of these algorithms:

1. They evaluate every chromosome against all states, even when it is unlikely that a chromosome will distinguish a state from others. This results in significant unnecessary computation, particularly for large FSMs.
2. GA are sequential and execute on a single thread, resulting in limited parallelism and underutilisation of modern multi-core processors.

ASPR addresses these limitations by combining evolutionary search with structured state-space refinement. Like genetic algorithms, ASPR maintains a population of chromosomes (referred to as a genome), which are iteratively selected, mutated, and evolved using a fitness function. However, unlike standard GAs, ASPR introduces the concept of blocks, which partition the state space into smaller subsets. Each block maintains its own local context, which controls how chromosomes are selected and applied states. This block-based design enables more targeted evaluation of chromosomes, Improved computational efficiency, and better scalability for large FSMs.

Three variants of ASPR are proposed:

- ASPR Standard
- ASPR Full
- ASPR Hybrid

In addition, parallel versions of each algorithm are developed, denoted by the prefix “P-” (e.g., P-ASPR Standard). These are multithreaded variants that enable concurrent processing of multiple blocks.

The following subsections describe the high-level design of the ASPR family. Parallel variants are not described separately, as they retain the same core logic as their sequential counterparts, differing only in execution strategy. Parallel variants are not covered separately, as they have the same core logic as their sequential counterparts, differing only in execution strategy.

3.1.1 ASPR Standard

This algorithm was the first working version of ASPR created and is the foundational algorithm in the ASPR family. It is later used as a basis to create the other algorithms in this family. It is designed to reduce unnecessary computation by avoiding the global application of chromosomes across all states.

ASPR Standard works by dividing the FSM into blocks, where each block contains a subset of states together with the information needed to continue searching for UIOs. Instead of applying all chromosomes to the entire FSM, chromosomes are only applied to the states inside the current block. This allows the search to focus on smaller groups of still-indistinguishable states. Chromosomes are also limited by a maximum depth.

The algorithm begins with a single block containing all states. As chromosomes are applied, states are grouped according to the output sequences they produce, which creates new refined blocks. A fitness function is used to favour chromosomes that separate states more effectively, and each block evolves its own genome through selection and mutation.

When a block is reduced to a single state, its IO trace is treated as a candidate UIO and is validated. If it is not unique, the block is expanded further, otherwise a UIO is recorded for that state. This process continues until the block queue becomes empty or the maximum number of generations is reached.

The main advantage of ASPR Standard is that it reduces the computation required to apply a chromosome by working on smaller subsets of states rather than the full FSM each time. However, validating candidate UIOs during block generation adds overhead.

3.1.2 ASPR Full

This algorithm is very similar to ASPR Standard but with a few differences. Blocks maintain both the original states and their corresponding transitioned states, allowing subsequent chromosomes to be applied incrementally from the result of previous input sequences rather than restarting from the initial state. This is so that chromosomes with a high enough fitness score are applied iteratively instead of just once, to better partition states and find UIOs. This does not result in excessively long input sequences, as chromosomes with a depth higher than the max depth are never added to a genome. To implement the logic in ASPR Full, one more cache is needed than in ASPR Standard.

In this approach, blocks which have only one state are ‘terminated’. After all blocks in the block queue have been processed, the IO sequence of each terminated block is validated. If the IO is deemed to be a valid UIO, then a UIO is recorded for this state. Otherwise, if another state produces the same IO sequence, then this is added to the set of states of the block, and the block is added to the block queue so that it can be processed again.

The advantages of validating UIOs after all blocks have been processed is that it reduces the computational recourses required, as validating UIOs is very computationally expensive. However, the drawback is that if the corresponding IO sequence is not a true UIO, additional refinement is required, which introduces overhead as the block must be reintroduced into the block queue and processed again.

3.1.3 ASPR Hybrid

ASPR Hybrid combines the strengths of ASPR Standard and ASPR Full by splitting UIO validation into two stages:

1. During block generation, candidate UIOs are checked using a cache. This provides a fast approximation but does not guarantee correctness.
2. After all blocks are processed, remaining candidate UIOs are validated in depth against all other states. This guarantees that UIOs are valid.

In both these processes, if a block with the same input/output trace already exists, the new states are merged into that block.

The reason for splitting the UIO validation into 2 is to balance between reducing the computational recourses required to constantly fully validate the candidate UIOs and reducing the overhead required to constantly add blocks to the block queue.

3.2 GA-Block

In GA, once the population changes multiple times in a genome, it becomes very different from what it was before, meaning that chromosomes that are like the ones in the genome that have a high fitness score may not be applied. GA-Block was designed to solve this issue and is a block-based approach inspired by ASPR and based on genetic algorithms.

Like in ASPR, the algorithm relies on a block queue which contains blocks. However, each block in this approach contains less information than blocks in ASPR variants. GA-Block only maintains the following:

- Input traces
- Genome

When a block is processed, the chromosomes in the genome are applied to every state in the FSM. If a block contains only one state, then a UIO is recorded for that state. Unlike ASPR, since chromosomes are applied to every state in an FSM, UIO validation is not needed as UIOs are definitely valid. Just like in GA, mutation and selection are applied to chromosomes in the genome to create a new genome. A new block is created and added to the block queue.

While this algorithm is sequential, it is the pre-cursor to GA-P-Block, which is parallel and uses the block-based approach to allow multiple genomes to be evaluated at once.

3.3 GA Table

To process each chromosome in a genome, GA requires the following triple nested loop:

```
For each chromosome  $c \in g$  do
  For state  $s \in S$ :
    For input  $x \in X$ :
      fsm.transition_with_state(...)
```

This requires multiple loops and so requires programming language interpreter overhead, multiple function calls, iterative object access. Consequently, GA scales poorly because the computational cost becomes significant as the number of states, chromosomes, or input length increases.

On the other hand, GA Table uses a vectorized approach by leveraging the NumPy library. Instead of iterating over states and inputs individually, the FSM is first compiled into transition and output tables, enabling constant-time indexed lookups. Chromosomes are then evaluated in batches using array operations, allowing multiple states and chromosomes to be processed simultaneously. GA uses dictionaries to group IOs, whereas GA Table uses optimized NumPy array operations.

GA Table achieves substantially better scalability, particularly for large FSMs and populations, while maintaining the same underlying search strategy as the standard GA.

3.3.1 GA Table Variants

Multiple variants of GA Table were created, each for different purposes. The first variant created was GA Table GPU. This is implemented in the same way as GA Table but uses PyTorch instead of NumPy, so that the algorithm can run on a CUDA enabled GPU instead of on CPU. This algorithm was designed to take advantage of the efficient design of GA Table and run the algorithm on GPU at the same time.

For Both GA Table and GA Table GPU, block and parallel versions were created. These algorithms were named the following:

GA Table Block

GA Table GPU Block

3.4 Parallel Variants

To further improve scalability, parallel versions of each algorithm proposed in this section are developed. The names of these variants are denoted by the prefix “P”, for example:

P-ASPR Standard

GA P-Block

GA Table P-Block

The parallel variants extend the corresponding sequential algorithms by enabling concurrent processing of blocks. Instead of processing blocks sequentially, multiple blocks are evaluated simultaneously using a pool of worker threads.

The parallel design includes a shared work queue containing blocks to be processed, multiple worker threads that dequeue and process blocks concurrently, synchronisation mechanisms (e.g., locks) to protect shared data structures.

This design allows the algorithm to exploit modern multi-core architectures, to significantly improve throughput. However, parallelization requires synchronization overhead due to locking.

3.4 Summary

This chapter has reviewed the high-level designs of all the newly proposed algorithms in this project. The next chapter will go over the low-level designs of the algorithms in more detail.

4 Low-Level designs

4.1 Genetic Algorithm

This section describes the low-level design of the standard Genetic Algorithm (GA) used in this project for constructing UIOs.

The algorithm maintains two main data structures throughout execution. The first is a UIO map, which stores the UIO found for each state. The second is a genome, which stores chromosomes together with their current fitness values. The set of chromosomes in the genome is called the population.

The initial population is created by generating random chromosomes of varying lengths from 1 up to a maximum chromosome depth. Duplicates are removed by storing chromosomes in a set, and the final population is sorted before being returned.

At the beginning of the algorithm, the genome is initialized from the given population. Each chromosome in the starting population is inserted into the genome and assigned an initial fitness score of 1. This means that all chromosomes begin with equal priority before any evaluation has taken place. The algorithm then runs repeatedly over generations until either all states have been assigned a UIO or the maximum number of generations has been reached.

In each generation, every chromosome in the population is applied to every state in the FSM. Each input symbol in the chromosome is applied one by one using the FSM transition function. As this happens, the output symbols produced are collected in sequence. Once the chromosome has finished executing, the resulting output sequence is stored as a tuple.

The algorithm then groups states according to the output sequence they produced for that chromosome. This grouping is stored in a dictionary whose keys are pairs of the form (chromosome, output sequence) and whose values are the states that produced that output sequence. In effect, this records how well a chromosome separates the states of the FSM.

This grouping is then used to calculate fitness scores and identify UIOs. For this implementation, the fitness scores depend on the size of the state group corresponding to a given output sequence. Chromosomes that leave fewer states grouped together are considered better, since they distinguish states more effectively. The computed score is also multiplied by the chromosome's previous fitness value stored in the genome, so that chromosomes which have performed well in earlier generations continue to be favoured over those who have performed worse. Since the computed score is always at most 1, the fitness score of each chromosome declines over time, which gives newer chromosomes an advantage and means newer chromosomes often get a higher fitness score than old ones.

At the same time, the algorithm checks whether a UIO has been found. If a chromosome produces an output sequence that is generated by exactly one state, then that input/output pair uniquely identifies that state and a UIO for that state is inserted into the UIO map, provided that the state does not already have a recorded UIO.

After all chromosomes in the current population have been evaluated, the genome is updated with the newly computed fitness values. If all states now have UIOs, the algorithm terminates and returns the UIO map. If not, a new population is constructed using selection, crossover, and mutation. This occurs in the following process:

1. First, the population is sorted in descending order of fitness, and the top half of the chromosomes are retained directly and form the parent pool.
2. Another chromosome is created by selecting two parents at random from the population and combining them using crossover. A crossover point is chosen, and the child chromosome is formed by taking the prefix from one parent and the suffix from the other.

3. There is a 1 in 10 chance that mutation occurs. This means that one randomly chosen input in the child chromosome is replaced with another valid input from the FSM.

Steps 2 and 3 are repeated until the new population reaches the required population size, at which point the current population of the genome is replaced, with new chromosomes having a fitness score of 1. After this, the next generation begins and the algorithm keeps looping until UIOs have been found for all states.

4.2 ASPR Family

This subsection describes the low-level design of algorithms in the ASPR family, including how blocks are processed, how chromosomes are evaluated, and how UIOs are identified.

4.2.1 ASPR Standard

ASPR standard contains a structure called a block, which maintains the following information:

- A set of states
- Input traces
- Output traces
- Genome

This algorithm requires the following information:

- A queue is used to store blocks that need to be processed. This is called a ‘block queue’.
- A map is used to store previously seen blocks, indexed by their corresponding input and output traces. This is called a ‘block map’, and allows blocks with identical traces to be merged, so that the block isn’t unnecessarily reprocessed.
- A cache called a ‘Input/output cache’ is also maintained, which stores mappings from input/output traces to the set of states that produce them. This is used later during a phase called UIO validation.
- A dictionary that stores UIOs. This is called a ‘UIO map’ and is indexed by state id, with its values containing the

The algorithm repeatedly removes block from the queue and processes them, with each iteration of block removal called a generation. Initially, a queue contains only the starting block. For each chromosome, the FSM is simulated starting from each state in the block. After chromosome is applied, both the input sequence and the corresponding output sequence are recorded and are stored in an Input/output cache, so that it is known which states produce a given input/output trace. This can be used later to quickly validate candidate UIOs, which will be discussed further down. States are grouped based on the output sequence that they produce, and this grouping is used to form new blocks, where each block represents a set of states that are indistinguishable from each other thus far. Child blocks inherit all their attributes, apart from the set of states, from their parent.

For each group of states, the blocks map is used to check if another block has the same input/output trace. If another block does have this, the states in this group are merged with the existing block, otherwise a new block is created containing the group of states, with each new block inheriting its information from its parents. A fitness score for each chromosome is determined by how well it partitions the states within the block and is calculated by comparing the number of states in the block with the number of states that produce identical output sequences for that chromosome. Chromosomes that result in a higher number of distinct outputs are given higher fitness scores, as they partition states better. Shorter input sequences are given a higher score.

Next, selection happens, meaning only the chromosomes with the highest fitness values are retained in the block’s genome, while the other chromosomes are replaced using crossover and mutation. In this stage, two parents are selected from the parent pool. A crossover point is chosen, after which a child chromosome is constructed by taking the prefix from one parent and the suffix from the other. The

chromosome is then added to the genome. A new block is then created using the updated genome and is added to the block queue.

If a block contains more than one state, then no candidate UIOs are recorded, and the block is added back to the queue for further processing. If a block contains exactly one state, then its input/output traces are treated as a candidate UIO. This candidate is then validated to ensure that it uniquely identifies the state via the following process:

1. The Input/output cache is used to check to determine how many states produce the same trace.
2. If only one state is found, a full verification step is performed by applying the chromosome to every state in the FSM and checking that no other state produces the same output sequence.

If at either stage another state produces the same IO sequence, the state is added to the block's set of states, and the block is added back to the queue for further refinement, and another generation of the algorithm runs. If the validation is successful, the input/output sequence is recorded as a UIO for that state. Once either all states have been assigned a UIO, the block queue becomes empty, or the maximum number of generations have occurred, then the algorithm stops and the UIO sequences that have been found are returned.

4.2.2 ASPR Full

This algorithm mostly follows the same structure as ASPR Standard, and so only the differences are described here.

ASPR Full contains a different block structure than ASPR Standard, as more information is needed for this algorithm. The structure is as follows:

- A set of starting states
- A set of transitioned states
- Input traces
- Output traces
- Genome
- Whether or not the block is terminated

During processing of chromosomes, after the current input trace has been applied, the algorithm keeps track of the transitioned state that has been reached from the original state and stores it in the block. Chromosomes in the same block are applied from these transitioned states instead of starting again from the original states. This allows the algorithm to process a longer input sequence while only doing computation of transitions for the chromosome that is applied to the transitioned state. Since the block maintains the full input trace as well as the starting states, new chromosome longer chromosomes to be processed more. This also improves block merging, as blocks are matched not only on identical traces but also on consistent state progression, reducing incorrect merging and avoiding redundant computation.

The last difference is in how candidate UIOs are handled. In ASPR Standard, when a block contains one state, the candidate UIO is fully validated immediately. In ASPR Full, this does not happen straight away and instead, the block is marked as terminated. After the block queue becomes empty, the terminated blocks are validated. If the trace really is unique, then the UIO is recorded. If more than one state produces the same trace, those states are added to the block, the block is unterminated and is returned to the queue so that it can be processed again.

4.2.3 ASPR Hybrid

Since this algorithm is very structurally similar to ASPR Standard, only the differences are described here.

In ASPR Standard, when a block contains one state, the candidate UIO is fully validated. In ASPR Full, a candidate UIO is fully validated only after all blocks have been processed. ASPR Hybrid aims to strike

a balance by splitting the validation into 2 parts and carrying out each part separately. The process is as follows:

After a block is processed, candidate UIOs are immediately validated using the Input/output cache. Recall from section 4.2.1 that the Input/output cache is used to store mappings from input/output traces to the set of states that produce them. is used to check to determine how many states produce the same input output trace. If only one state has produced a given input/output trace, that trace is recorded as a potential UIO for that state. Since different blocks contain different chromosomes and states, the cache may not hold all states that produce an input/output sequence, and so it does not guarantee that the UIO is correct. If more than one state produces the same input output trace, these states are added to the set of states in the block. This allows the algorithm to refine blocks for UIOs without doing expensive computation.

After the block queue becomes empty, ASPR Hybrid then performs a second validation stage. In this stage, every potential UIO is validated by applying the input sequence part of the UIO to all states in the FSM. If only one state produces the corresponding output trace, a UIO is recorded for that state, and the input/output sequence is removed from the potential UIOs. If more than one state produces the output trace, then the input/output sequence is removed from the potential UIOs, those states are added back into the remaining states set, and a block is created again for that trace so that it can be processed further.

4.3 GA-Block

GA-Block is implemented as a hybrid between a standard genetic algorithm and a block-based search strategy. Unlike ASPR, blocks do not store subsets of states as this is not needed. Instead, each block maintains the following information:

- A genome (mapping chromosomes to fitness scores)
- Input traces corresponding to chromosomes that have been explored.

A queue is used to manage blocks that are yet to be processed. Initially, the queue contains a single block whose genome is constructed from the initial population.

Each iteration of this algorithm is called a ‘generation’ and involves removing a block from the queue and processing it. During simulation, each chromosome in a block’s genome is applied to all the states in an FSM and states are grouped according to the output sequence. A fitness score is assigned to each chromosome based on how well it separates states. This is determined by measuring how many states produce identical output sequences. Chromosomes that result in smaller groups (i.e., better separation) are given higher fitness scores. Shorter chromosomes are also slightly favoured to bias the search towards shorter UIOs. Additionally, a generation-based scaling factor is applied so that earlier generations have a greater influence on fitness compared to later ones.

During block processing, UIOs are identified directly from the grouping of the states. If, for a given chromosome, an output sequence is produced by exactly one state, then that input/output pair is recorded as a UIO for that state. The identified state is then removed from the set of remaining states.

After all chromosomes in the block have been evaluated, a new generation of chromosomes is created using selection and crossover. Chromosomes with the highest fitness scores are retained, while others are replaced by successful chromosomes.

Next, crossover and mutation, which means that two parents are selected from the parent pool. A crossover point is selected, and a child chromosome is formed by combining the prefix of one parent with the suffix of the other. These chromosomes are added to the genome. A new block is then created using the updated genome and is added to the block queue.

The algorithm continues until one of the following conditions is met:

- All states have been assigned a UIO.
- The block queue becomes empty.

- The maximum number of generations is reached.

At termination, only the valid UIOs that have been discovered are returned.

4.4 GA-Table

GA Table has a similar strategy as GA but evaluates chromosomes differently. Before the algorithm begins anything, it converts the FSM into a compiled table representation. This allows chromosome evaluation to be carried out in parallel using NumPy arrays and table lookups. This is to reduce interpreter overhead and improves scalability compared to GA.

Before the search begins, the FSM is compiled into a table-based representation. This compiled FSM contains the following:

- state transition table, which is a mapping from each combination of starting state index and input index to a transitioned state.
- output transition table, which is a mapping from each combination of starting state index and input index to an output.
- A mapping from each input symbol to an integer index
- A mapping from output indices to output symbols.
- A mapping from state indices back to state identifiers. This

The above preprocessing step is important because it allows transitions and outputs to be retrieved by direct indexing rather than by repeatedly calling FSM methods during chromosome evaluation.

A dictionary called the genome is then created, which maps each chromosome to its current fitness score. Initially, every chromosome in the starting population is assigned a fitness of 1.0. A second dictionary, called the UIO map, is also maintained, which stores the UIO found for each state, indexed by state id. Each cycle of the algorithm is called a ‘generation’.

At the start of each generation, the algorithm ensures that every chromosome currently in the population has an entry in the genome. The chromosomes are then converted into a matrix form. Since chromosomes may have different lengths, a fixed-size matrix is created whose number of rows equals the number of chromosomes and whose number of columns equals the maximum chromosome length in the current population. Each row corresponds to one chromosome, and each gene is replaced by its integer input index. Unused positions are padded with -1. A second array stores the true length of each chromosome.

The algorithm also creates a matrix of current states, which stores, for each chromosome, the current FSM state index for every state in the machine. Initially, this is just all FSM states replicated once for each chromosome. Another array stores the output sequence produced when each chromosome is applied to each state. Its dimensions are number of chromosomes, number of states, and maximum chromosome length. This means that the entire batch of chromosome executions can be stored and processed together.

For each gene position, the algorithm first determines which chromosomes are still active at that position by comparing the chromosome lengths against the current depth. Only those chromosomes are processed. For the active chromosomes, the current gene is taken from the chromosome matrix, and the next outputs and next states are obtained directly from the compiled output and state transition tables using NumPy indexing. The produced outputs are written into the output array, and the current states are updated accordingly. This continues until all positions in the chromosomes have been processed. In effect, this replaces the triple nested loop of the standard GA with batch operations over arrays.

Once all chromosomes have been simulated, fitness values are calculated. The previous fitness values are first copied so that some information from earlier generations can be retained. The new fitness values are then computed separately for chromosomes of each distinct length. For a given chromosome length, the corresponding output sequences are extracted from the output array. These output sequences are then encoded into integer signatures. This is done by treating each output sequence as a base-b number, where

b is one greater than the number of distinct outputs. As a result, output sequences that are identical receive the same signature value.

For each chromosome, the signature values are grouped using `np.unique`, which returns the distinct groups, an inverse mapping, and the size of each group. This allows the algorithm to determine how well the chromosome partitions the states. If many states produce different output sequences, then the chromosome has separated the states well. If many states still produce the same output sequence, then the chromosome has poor distinguishing ability. The partition score is therefore computed from the group sizes, and the number of singleton groups is also counted. The final fitness score combines the partition score, the singleton bonus (which is bonus weight is used in the fitness function to reward chromosomes that isolate states on their own, making them more likely to lead to valid UIOs), and part of the previous generation's fitness. This means that good chromosomes are rewarded both for general partitioning quality and for directly isolating states.

During this same stage, whenever a group has size one, the algorithm obtains the state index of that singleton state and reconstructs the corresponding output sequence. If that state has not already been assigned a UIO, the chromosome and output tuple are stored in the UIO map for that state. Since each chromosome in GA Table is evaluated against all states in the FSM, this identification is already global. Therefore, unlike ASPR, GA Table does not need an additional validation phase.

After all chromosomes have been evaluated, the new fitness values are written back into the genome. If UIOs have been found for all states, the algorithm terminates immediately and returns the UIO map. Otherwise, a new genome is generated using a selection and crossover phase. The selection and crossover stage of the genome creation begins by sorting the population in descending order of fitness. The chromosomes which have fitness scores that are in the top 50% are copied directly into the next genome.

The rest of the new genome is created using crossover and mutation. Two parents are selected from the genome. A crossover point is selected, and a child chromosome is formed by combining the prefix of one parent with the suffix of the other. If the length of any the child chromosome exceeds the maximum allowed depth, it is truncated. Mutation is then applied with a fixed probability by replacing one gene in the child with a random input symbol. This process continues until the genome reaches the required size. The next generation then begins with this new genome.

The algorithm stops when one of three conditions is met:

- all states have been assigned a UIO.
- the maximum number of generations has been reached.
- the genome population becomes empty.
- At termination, the algorithm returns the UIO map containing all UIOs that were found.

GA Table was also implemented in PyTorch so that GPU can be used to allows parallel simulation of all chromosomes across all states simultaneously. However, to simulate chromosomes in parallel, a CUDA enabled GPU is required, otherwise the algorithm runs sequentially on CPU. The implementation in PyTorch follows the same structure but replaces NumPy functions with PyTorch functions.

4.5 Parallel Variants

The parallel variants created in this project are the algorithms whose names that have a 'P-' prepended to either 'ASPR' or 'Block'. These algorithms are parallel versions of their corresponding block-based algorithms. Their main logic is not changed from them, and the main difference is in how the blocks are processed. In the sequential versions, the block queue is processed one block at a time whereas in the parallel versions, multiple blocks can be processed at the same time by different worker threads.

1. The overall structure for parallelism is the same in all variants and follows the following procedure:
2. First, an initial block is created and added to a shared work queue. A `ThreadPoolExecutor` is then created, and worker threads repeatedly remove blocks from the queue and process them. If

processing a block creates more blocks, these are added back into the shared queue so that they can also be processed.

This above procedure continues until either all UIOs have been found, the queue becomes empty, or the generation limit is reached.

Since multiple threads are running at the same time, it is inevitable they attempt to access or update the same data structures at the same time causing a race condition. Because of this, locks are used to protect the shared structures. These include the UIO map, the set of remaining states, the block dictionary, the Input/output cache, and in some cases the fitness map or the generation counter. This is needed so that updates are done correctly and so that different threads do not interfere with each other.

Another difference is that in parallel variants, the order in which blocks are processed isn't fixed. In the sequential versions, this order follows the queue directly, since only one block is processed at a time. In the parallel versions, this depends on which worker thread becomes available first so the processing order may change from run to run. However, the main aim is still the same as in the sequential versions, which is to use the same block-processing logic, but to explore several blocks at once instead of only one.

4.6 Summary

This section has described the low-level structure of each algorithm that was used in the project. Next, the empirical study will be reviewed, where each of these algorithms are evaluated and compared across multiple FSMs.

5 Empirical Study

5.1 Experimental Setup

All experiments were executed through a sandbox environment used for testing different algorithms on FSMs. All algorithms that were executed had a maximum chromosome depth of 6 and population size of 10. The chromosome depth refers to the number of genes in the chromosome, whereas the population refers to the set of chromosomes in a genome. The same overall framework was used across all experiment groups so that the comparisons remained consistent.

All experiments were run on a Lenovo IdeaPad S540-15IWL system running Microsoft Windows 11 Home (version 10.0.26100). The system was equipped with an Intel Core i5-8265U CPU (4 cores, 8 logical processors, base frequency 1.60 GHz), Intel® UHD Graphics 610, and 8 GB of RAM.

All algorithms were executed on the CPU. No CUDA enabled GPU was available, and therefore all results reflect CPU-based execution only.

5.2 FSM Data

Each FSM used in the experiments was stored in KISS2 format, which is a standard format used to represent FSMs in a compact way [22]. This makes it possible to rerun experiments on the same FSM instances rather than regenerating different machines each time. This is necessary as FSMs with the same number of inputs, outputs and states can have different behaviour. Storing FSMs ensures that different algorithms can run on the exact same FSM.

Multiple experiments were conducted, each defined by a set of FSMs and a group of algorithms. The experiments were carried out on FSMs from three FSM repositories, referred to as Repository A, Repository B, and Repository C.

Repository A contains FSMs with 250, 500, 750, 1000, and 1500 states. The input and output alphabet sizes were selected from the set {10, 50, 100, 200, 350, 700}. Constraints were applied to exclude unnecessarily extreme combinations. In total, this repository contains 26 FSMs.

Repository B contains FSMs with 500, 1000, 2000, 4000, 7000, 10000, and 15000 states. The input and output alphabet sizes were selected from the set {10, 50, 100, 350, 650, 1000}, with additional constraints used to avoid extreme configurations. This repository contains 65 FSMs in total.

Repository C uses the same state sizes and input/output alphabets as Repository B, but applies a less restrictive set of constraints, resulting in a larger and more diverse set of FSMs. This repository contains 154 FSMs in total.

Before plotting the results, each algorithm was executed at least 3 times for each FSM, and the results were averaged out to reduce the impact of outliers and prevent them from skewing the results. Although an algorithm in an experiment can be run for multiple iterations, each algorithm was not run again until all other algorithms were run first for that experiment. This is to prevent algorithms from being executed multiple times back-to-back so that other events such as a sudden spike in CPU usage would not have too much of an impact on one algorithm and the impact of such events can be spread out across multiple algorithms.

5.3 Evaluation Approach

The experiment executes each of the seven experiment groups six times and in each experiment the results for each FSM are written to a txt file. The results for each FSM and for each algorithm are also plotted on graphs.

The plotting stage generates comparisons mainly in three different ways. First, it fixes the number of inputs and outputs and varies the number of states. Second, it fixes the number of states and outputs and varies the number of inputs. Third, it fixes the number of states and inputs and varies the number of outputs. This makes it possible to examine scalability separately with respect to each of the three main FSM dimensions.

The goal of the study was to separate algorithmic effects from implementation effects where possible. For this reason, the experiments did not only compare different algorithm families but also compared sequential and parallel versions of the same method, block and non-block variants of the same approach, and compiled and uncompiled versions of the same table-based implementation. This makes the empirical study more informative, since improvements can be related more clearly to the design decision that caused them.

The experiments were designed to assess the overall effectiveness of all the proposed algorithms, on multiple metrics including the following:

- How the time taken for the algorithm to complete changes as the numbers of states, inputs, and outputs increase.
- The number of UIOs found for each algorithm.
- The median length of UIOs
- The accuracy of UIOs

5.4 Results

5.4.1 ASPR Family

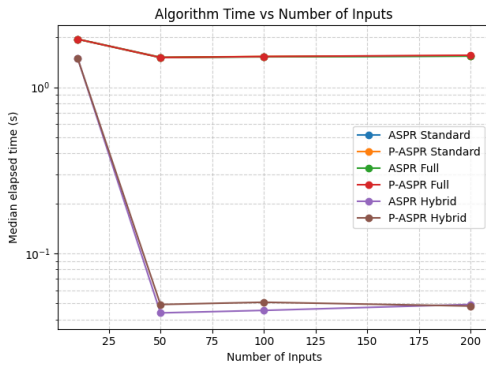


Fig 1: Results for FSMs with 250 states and 100 outputs

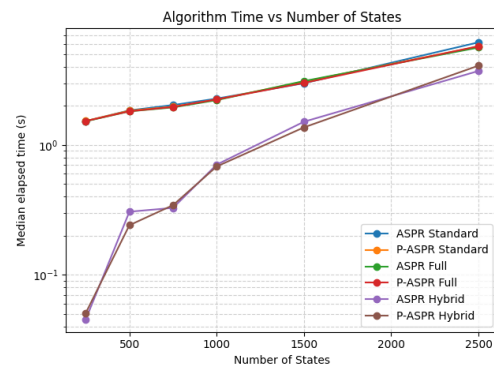


Fig 2: Results for FSMs with 100 inputs and 100 outputs

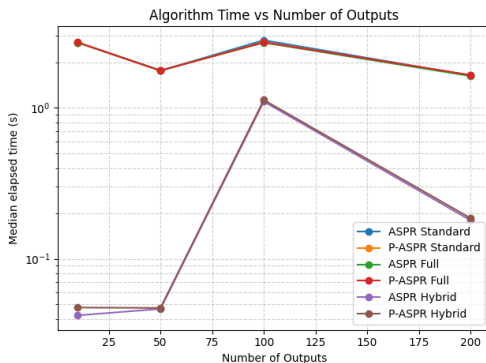


Fig 3: Results for FSMs with 100 inputs average out across all states

In this subsection, the performance of all sequential ASPR variants are evaluated and compared.

ASPR Full was slightly faster and more scalable than ASPR Standard, and ASPR Hybrid was faster and scaled better than both ASPR Standard and ASPR Full by multiple magnitudes. The results in from the experiments carried out for the ASPR family were very clear, consistent and were true across different numbers of states, inputs and outputs. As an example, take the following results:

FSM with 250 states, 100 inputs and 100 outputs	
Algorithm	Average Elapsed time (s)
ASPR Standard	0.19856
ASPR Full	0.16924
ASPR Hybrid	0.04533

Fig 4: Results for FSMs with 250 states, 100 inputs and 100 outputs

FSM with 2500 states, 100 inputs and 100 outputs	
Algorithm	Average Elapsed time (s)
ASPR Standard	16.24231
ASPR Full	16.12436
ASPR Hybrid	3.2083

Fig 5: Results for FSMs with 2500 states, 100 inputs and 100 outputs

The average length of UIOs, number of UIOs found was always identical, and all the UIOs were always correct.

5.4.2 GA Block

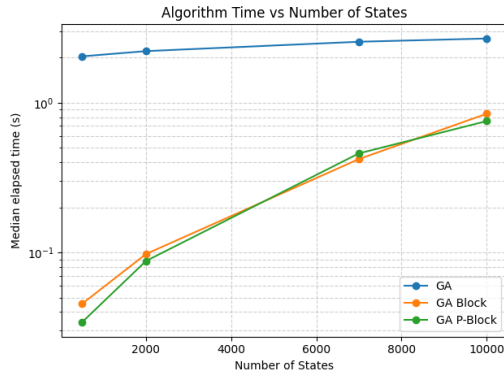


Fig 6: Results for FSMs with 100 inputs and 100 outputs

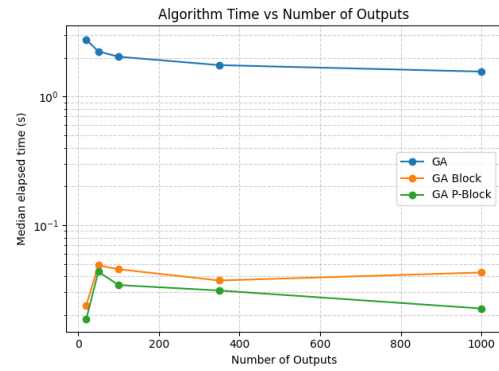


Fig 7: Results for FSMs with 500 states and 100 inputs

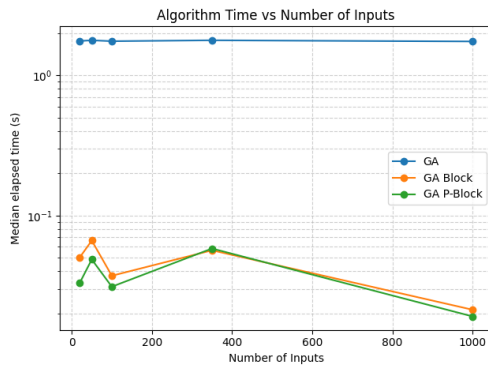


Fig 8: Results for FSMs with 500 states and 350 outputs

5.4.3 GA Table

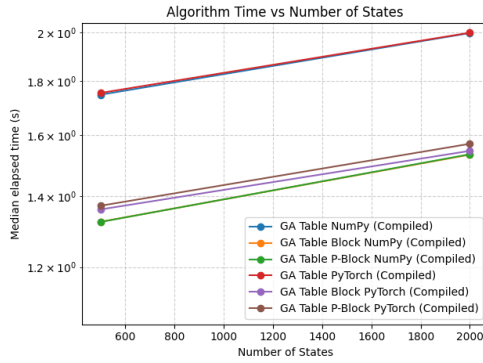


Fig 9: Results for FSMs with 350 inputs and 350

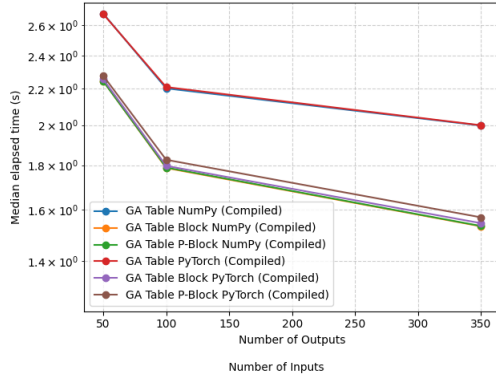


Fig 11: Results for FSMs with 2000 states and 350 outputs

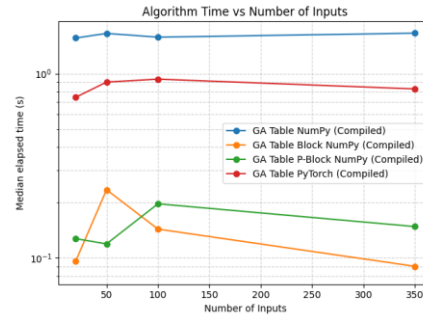


Fig 10: Results for FSMs with 7000 states and 350 outputs

In this experiment, both NumPy and PyTorch versions were compared. Compiled versions are ones where the tables are precomputed.

5.4.5 Parallel Variants

ASPR Standard and P-ASPR Standard are very similar in their execution times, with ASPR Standard performing on average 0.31% faster than P-ASPR Standard for an FSM with 250 states, 100 inputs, 100 outputs, and on average 2.49% faster for an FSM with 2500 states, 100 inputs and 100 outputs.

ASPR Full and P-ASPR Full are also very similar, with ASPR Full performing on average 0.41% faster than P-ASPR Full for an FSM with 250 states, 100 inputs and 100 outputs, and performing on average 3.36% faster than P-ASPR for an FSM with 2500 states, 100 inputs and 100 outputs.

ASPR Hybrid and P-ASPR Hybrid showed similar results in terms of their speed, but much more significant differences in their scalability. ASPR Hybrid was 0.41% faster than P-ASPR Hybrid for an FSM with 250 states, 100 inputs and 100 outputs, and 8.99% faster for an FSM with 250 states, 100 inputs and 100 outputs.

The results for the performance of all the above algorithms on these FSMs were consistent with other FSMs.

For the ASPR family, the sequential versions always outperformed the parallel versions. While the parallel versions were designed for scalability, they had the opposite effect, perhaps because of the computational overhead of using multiple locks and having to schedule blocks.

5.4.6 Summary

This section went over the results of several of the proposed algorithms and made comparisons

6 Conclusions

The initial proposal focused on methods for estimating the existence of UIO sequences in resource-constrained environments by using confidence scores. However, during the project, this direction was revised in favour of developing and evaluating standard UIO construction algorithms. This change was motivated by the need for certainty in practical testing scenarios, where testers require guaranteed correctness rather than probabilistic estimates, because the whole point of UIOs is that it allows a tester to be certain that their implementation matches a specification. As a result, the project's objective shifted towards designing efficient and parallel UIO construction algorithms for large FSMs.

Most importantly, this paper found that GA-Block and GA-P-Block perform much better than standard GA even for very large FSMs, but scales worse, making it less effective for huge FSMs with massive numbers of inputs, outputs, states.

7 References

- [1] E. F. Moore, “Gedanken-experiments on sequential machines,” in *Automata Studies*, C. E. Shannon and J. McCarthy, Eds. Princeton, NJ, USA: Princeton University Press, 1956, pp. 129–153.
- [2] T. S. Chow, “Testing software design modelled by finite-state machines,” *IEEE Trans. Softw. Eng.*, vol. SE-4, no. 3, pp. 178–187, May 1978.
- [3] D. Lee and M. Yannakakis, “Testing finite-state machines: State identification and verification,” *IEEE Trans. Comput.*, vol. 43, no. 3, pp. 306–320, Mar. 1994.
- [4] D. Lee and M. Yannakakis, “Principles and methods of testing finite-state machines: A survey,” *Proc. IEEE*, vol. 84, no. 8, pp. 1089–1123, Aug. 1996.
- [5] M. Sipser, *Introduction to the Theory of Computation*, 3rd ed. Boston, MA, USA: Cengage Learning, 2012.
- [6] A. Petrenko and N. Yevtushenko, “Testing from partial deterministic FSM specifications,” *IEEE Trans. Comput.*, vol. 54, no. 9, pp. 1154–1165, Sep. 2005.
- [7] R. M. Hierons et al., “Using formal specifications to support testing,” *ACM Comput. Surv.*, vol. 41, no. 2, pp. 1–76, 2009.
- [8] K. Naik, “Efficient computation of unique input/output sequences in finite-state machines,” *IEEE/ACM Trans. Netw.*, vol. 5, no. 4, pp. 585–599, Aug. 1997.
- [9] F. C. Hennie, “Fault-detecting experiments for sequential circuits,” in *Proc. 5th Annu. Symp. Switching Circuit Theory and Logical Design*, Princeton, NJ, USA, 1964, pp. 95–110.
- [10] A. V. Aho, A. T. Dahbura, D. Lee, and M. U. Uyar, “An optimization technique for protocol conformance test generation based on UIO sequences and the rural Chinese postman tour,” *IEEE Trans. Commun.*, vol. 39, no. 11, pp. 1604–1615, Nov. 1991.
- [11] D. P. Sidhu and T. K. Leung, “Formal methods for protocol testing: A detailed study,” *IEEE Trans. Softw. Eng.*, vol. 15, no. 4, pp. 413–426, Apr. 1989.
- [12] J. Tretmans, “Model based testing with labelled transition systems,” in *Formal Methods and Testing*. Berlin, Germany: Springer, 2008, pp. 1–38.
- [13] H. Ural, X. Wu, and F. Zhang, “On minimizing the lengths of checking sequences,” *IEEE Trans. Comput.*, vol. 46, no. 1, pp. 93–99, Jan. 1997.
- [14] R. M. Hierons, “Minimizing the number of resets when testing from a finite state machine,” *Inf. Process. Lett.*, vol. 90, no. 6, pp. 287–292, 2004.
- [15] R. M. Hierons and H. Ural, “Generating a checking sequence with a minimum number of reset transitions,” *Autom. Softw. Eng.*, vol. 17, no. 3, pp. 217–250, 2010.
- [16] A. da Silva Simão and A. Petrenko, “Checking completeness of tests for finite state machines,” *IEEE Trans. Comput.*, vol. 59, no. 8, pp. 1023–1032, 2010.
- [17] M. Yao, A. Petrenko, and G. v. Bochmann, “Conformance testing of protocol machines without reset,” in *Protocol Specification, Testing and Verification XIII*, 1993, pp. 241–256.
- [18] G. Luo, A. Petrenko, and G. v. Bochmann, “Selecting test sequences for partially-specified nondeterministic finite state machines,” in *Proc. 7th IFIP Workshop on Protocol Test Systems*, Tokyo, Japan, 1994.
- [19] H. Ural and Z. Wang, “Synchronizable test sequence generation using UIO sequences,” *Comput. Commun.*, vol. 16, no. 10, pp. 653–661, 1993.
- [20] Q. Guo, R. M. Hierons, M. Harman, and K. Derderian, “Computing Unique Input/Output Sequences Using Genetic Algorithms,” in *Formal Approaches to Software Testing (FATES 2003)*, LNCS 2931, pp. 164–177, 2004.
- [21] R. M. Hierons and U. C. Türker, “Parallel Algorithms for Testing Finite State Machines: Generating UIO Sequences,” *IEEE Trans. Softw. Eng.*, vol. 42, no. 11, pp. 1077–1091, 2016.
- [22] Berkeley Logic Interchange Format (BLIF) and KISS2 Format, University of California, Berkeley.
- [Online]. Available: https://people.eecs.berkeley.edu/~alanmi/research/espresso/espresso_5.html

Appendix A - Project Proposal

Probabilistic Generation of UIO Sequences for Deterministic Finite State Machines in Resource-Constrained Environments.

Abstract

The proposed project aims to firstly investigate current techniques for generating Unique Input Output (UIO) sequences and then secondly develop a new algorithm that uses confidence scores to estimate whether a UIO exists for any state in a Deterministic Finite State Machine (DFSM). This new technique should be less computationally intensive and more efficient than current UIO generation techniques. This is important as current UIO generation techniques for medium and large FSMs can be computationally heavy, meaning that they are unsuitable for resource-constrained environments, such as Real-Time systems, embedded systems, and IoT devices.

1. Introduction

FSMs are formal models that are used to model systems that transition between different states while producing outputs. They can be used for system modelling, protocol designs, software verification and hardware testing.

While they don't always exist in all states/machines, UIO sequences are useful in verification and testing because they enable the testers to determine what state a system is in. It does this by using the input part of the UIO on the system. If the input output sequence generated by this matches the UIO, the tester knows that a system is in state s , otherwise the system is in a different state.

Deciding whether UIO sequences exist has been proved to be PSPACE-complete problem [13] meaning that a polynomial-time algorithm is not expected. When used with medium or large FSMs, current techniques for determining a UIO sequence are computationally expensive, meaning that they take a lot of time and physical resources [7],[8].

Derderian et al. [8] explored the automation of UIO generation using genetic algorithms. While their approach successfully discovered UIO sequences for FSMs and in certain cases faster than brute-force approaches, it still scaled poorly with FSM size. This suggests that even though technology is becoming increasingly automated, especially with the ongoing advancements in AI, UIO generation complexity and resource consumption will probably still remain an issue.

The proposed technique will aim to lower the resources required by UIOs by only exploring some of the states and generating probabilities that a system is in a state s , rather than a concrete yes or no answer. A conceptual rough design of the algorithm has already been created.

This project proposal will be split into the following sections: preliminaries, background, the proposed project; the program of work, and references. The background will contain a description of the related projects and existing techniques used for FSM state identification. The proposed project section will outline the project's aims and objectives, as well as the methodological approach used to create the technique and develop the testing software. The program of work section will present the project plan, which details the schedule for creating the dissertation as well as a Gantt chart. The references section will list all sources and research papers cited throughout this proposal.

2. Preliminaries

A Finite State Machine (FSM) is typically either a Mealy or Moore FSM. This project deals only with Mealy FSMs, which are modelled as a tuple $M = (S, X, Y, \delta, \lambda)$ where:

- $S = \{s_1, s_2, \dots, s_n\}$ is a finite set of states.
- $X = \{x_1, x_2, \dots, x_r\}$ is a finite set of inputs.
- $Y = \{y_1, y_2, \dots, y_v\}$ is a finite set of outputs.
- $\delta: S \times X \rightarrow S$ is a transition function.
- $\lambda: S \times X \rightarrow Y$ is an output function.

[1], [2]

FSMs are either deterministic or non-deterministic. A Deterministic Finite State Machine (DFSM) is a machine that upon receiving an input, transitions from one state to another and produces a single output [3]. A Non-Deterministic Finite State Machine (NDFSM) is a machine that upon receiving an input, may transition to one or many of several possible states [1]. This makes it more difficult to compute UIOs.

Juxtaposition is used to denote concatenation, such that if x_1, x_2 , and x_3 are inputs then $x_1x_2x_3$ is an input sequence. The symbol ϵ is used to denote an empty sequence. An input/output sequence consists of a sequence of input/output pairs of the form $x_1/o_1 \ x_2/o_2 \ \dots \ x_m/o_m$ [1].

Two states s_i and s_j are said to be equivalent if, for all possible input sequences, they produce identical output sequences [4]. A Unique Input Output (UIO) sequence is an input sequence that, when applied to a state, produces a unique output such that applying the sequence to any other state produces a different output [5], [6]. This can be used to identify a specific state.

3. Background

FSM state identification techniques in black-box environments have long been a topic of interest [1]-[4]. These methods include using a Distinguishing Sequence (DS), Unique Input Output sequence (UIO) [5], [8], [9]. When DSs are applied from any starting state, a distinct output sequence is produced for each state, which allows the tester to tell what state a machine is currently in [4], [10]. A Distinguishing Sequence can either be a Preset Distinguishing Sequence (PDS) or an Adaptive Distinguishing Sequence (ADS) [11]. While PDSs apply a fixed input, ADSs are formed by constructing sequences that adjust based on the observed outputs during execution [11]. ADSs are generally seen as more effective than PDS [11], can be exponentially shorter and can handle cases where uncertainty exists about the current state, making them more practical in

complex FSMs [6], [7], [11].

On the other hand, UIO sequences allow testers to identify the current state of a Finite State Machine (FSM) by applying an input that produces a unique output that differentiates it from other states. Identifying whether they exist is PSPACE-complete, meaning that a polynomial-time algorithm is not expected and so the resource requirements increase rapidly as the size of the FSM grows [13].

Current UIO generation methods include partition-based/state-partitioning methods [10]-[12] and brute force techniques. Brute-force methods grow exponentially with the input-sequence length, while partition-based/state-partitioning methods remain polynomial per step but still scale poorly for large FSMs [12]. This causes these approaches to have a high computational complexity, which makes them less effective in medium or large FSMs in constrained environments like embedded or real-time systems, where memory and processing resources are limited.

Several researchers have attempted to address these challenges. Hierons and Türker [7] introduced parallel algorithms to increase the efficiency of UIO generation, while Lehre and Yao [9] analyzed evolutionary algorithms for computing UIOs. Although these advances have found ways to make UIO generation more efficient, they are still computationally demanding and rely on exhaustive state exploration.

The proposed project introduces a probabilistic UIO generation technique that uses confidence scores to estimate whether a UIO sequence exists. Unlike conventional algorithms that require exhaustive state exploration, this method aims to only explore some of the states, allowing FSM testing and verification to be more efficient in resource-constrained environments.

4. The Proposed Project

4.1 Aims and Objectives

The aim of the project is to design an efficient probabilistic algorithm for discovering Unique Input/Output (UIO) sequences in Finite State Machines by only exploring some of the states and using confidence scores. To achieve the aim of this project, the following objectives are outlined:

- Research current methods used to discover UIO sequences.
- Design an initial algorithm for discovering Unique Input/Output (UIO) sequences. This is likely to not be optimal at the start and so will require optimization.
- Develop software used for testing the performance of the new technique as well as existing ones. This will be used to optimize the algorithm, as well as draw conclusions about its accuracy, efficiency, and scalability.

4.2 Methodology

The development of the project will follow the Waterfall methodology due to the natural linear style of the project. The exact details of how the project will be carried out will be outlined in

section 5. The project will consist of the following stages:

Stage 1: Research

- Conduct detailed research of FSM identification and UIO generation methods.
- Create performance benchmarks for comparing the new technique against existing algorithms.

Stage 2: Design

- Create flowcharts and pseudo-code for the new algorithm and testing software

Stage 3: Implementation

- Implement the probabilistic UIO generation algorithm, as well as the testing software by writing it in Python code.

Stage 4: Verification and Validation

- Measure algorithm performance and efficiency with different FSMs and compare results of different versions of the method.
- Debug and optimize the technique based on test results.

Stage 5: Deployment and Documentation

- Compile all the information gathered so far and document the whole project in a report.

5. Programme of Work

The project will begin on 27 October 2025 and will run until 24 March 2026, and it will consist of the following stages:

- Research (27 October – 19 November). This will involve reading and analyzing relevant studies and literature so that the UIO sequence discovery technique and the testing software can be made effectively
- Benchmark creation (19 November – 21 November). This will involve creating effective benchmarks that can be used for designing and testing the algorithm
- Design of new algorithm (21 November – 13 December). This will involve creating code or pseudocode for an efficient algorithm
- Design of testing software (13 December - 15 December). This will involve creating a plan and flowcharts for the testing software.
- Implementation of new algorithm (15 December – 19 December). This will start on the Christmas break and will involve creating the first version of the UIO sequence discovery algorithm in Python with unit tests.
- Implementation of testing software (19 December – 4 January). This will involve creating the full testing software
- Implementation of existing algorithms (4 January – 24 January). This will involve implementing 3 other existing methods for finding UIO sequences on Python so that they

can run on the testing software and be compared with the new technique.

- Optimization of new algorithm (24 January – 15 February). This will involve iteratively changing parameters and analyzing results of the algorithm.
- Evaluation (15 February – 24 March). This involves gathering all the data and results so far and writing a final report.

The overall schedule for the project is displayed by a Gantt chart in Figure 1 .

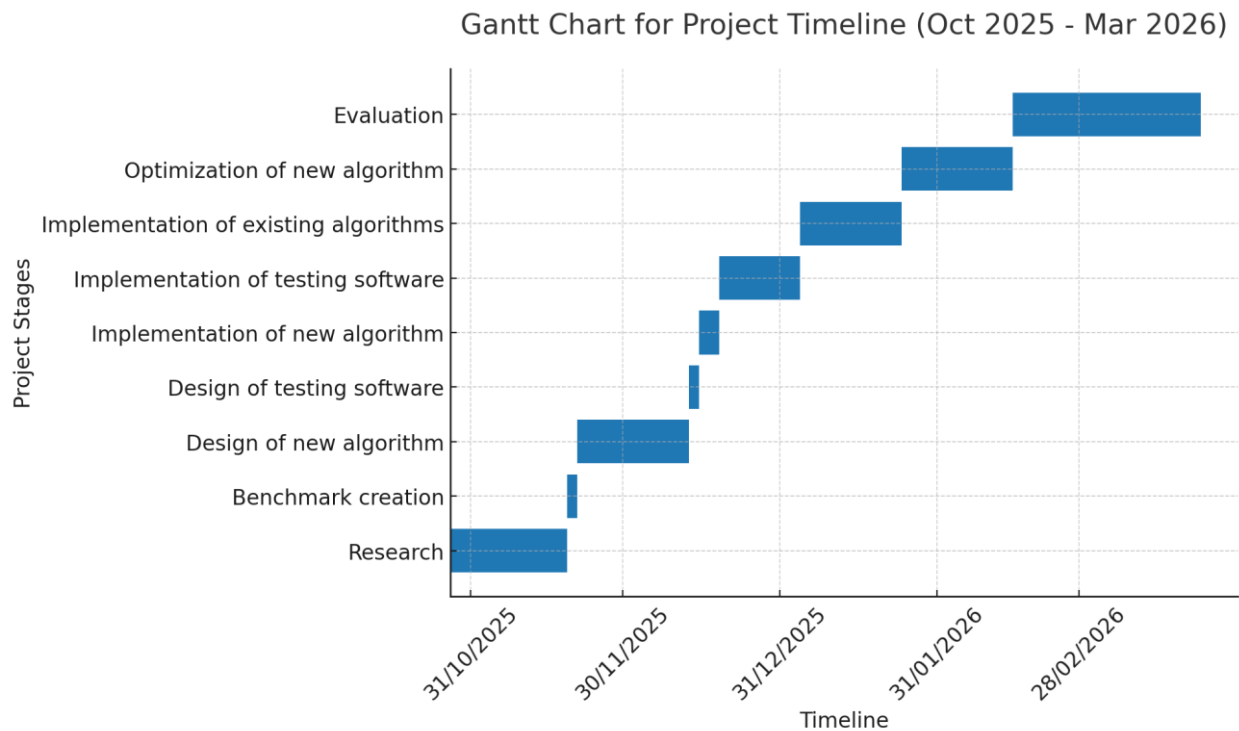


Figure 1: Project Schedule 2025-2026

6. References

1. A. Gill, *Introduction to the Theory of Finite-State Machines*. New York, NY, USA: McGraw-Hill, 1962.
2. Z. Kohavi, *Switching and Finite Automata Theory*, 2nd ed. New York, NY, USA: McGraw-Hill, 1978.
3. J. E. Hopcroft and J. D. Ullman, *Introduction to Automata Theory, Languages, and Computation*. Reading, MA, USA: Addison-Wesley, 1979.
4. E. F. Moore, "Gedanken-Experiments on Sequential Machines," in *Automata Studies*, C. E. Shannon and J. McCarthy, Eds. Princeton, NJ, USA: Princeton University Press, 1956, pp. 129–153.
5. K. K. Sabnani and A. T. Dahbura, "A Protocol Test Generation Procedure," *Computer Networks and ISDN Systems*, vol. 15, no. 4, pp. 285–297, 1988.
6. A. Petrenko, "Checking Experiments with Fault Detection Capabilities for Finite State Machines," *IEEE Transactions on Computers*, vol. 42, no. 10, pp. 1212–1226, Oct. 1993.
7. R. M. Hierons and U. C. Türker, "Parallel Algorithms for Testing Finite State Machines,"

White Rose Research Online, 2016.

8. K. Derderian, R. M. Hierons, and M. Harman, "Automated Unique Input Output Sequence Generation for Conformance Testing," Brunel University Research Archive, 2006.
9. P. K. Lehre and X. Yao, "Runtime analysis of the $(1+1)$ EA on computing unique input/output sequences in finite state machines," *Information Sciences*, vol. 259, pp. 510–524, 2014.
10. M. A. Ural, "State identification and verification of communication protocols," *Computer Communications*, vol. 13, no. 6, pp. 327–336, 1990.
11. S. Fujiwara, G. von Bochmann, F. Khendek, M. Amalou, and A. Ghedamsi, "Test sequence generation for finite state machines based on adaptive distinguishing sequences," *IEEE Transactions on Computers*, vol. 40, no. 4, pp. 421–425, 1991.
12. M. Yannakakis, "Testing finite state machines," *Proceedings of the 23rd Annual ACM Symposium on Theory of Computing (STOC)*, ACM, 1991, pp. 476–483.
13. D. Lee and M. Yannakakis, "Testing Finite-State Machines: State Identification and Verification," *IEEE Transactions on Computers*, vol. 43, no. 3, pp. 306–320, Mar. 1994.